

第3章 栈与队列

本章为 SmartSketch 项目自编的教学材料，仅用于抽取评测的开发与调试，不摘自任何教材或真实课程讲义。

3.1 本章导读

栈和队列是两种操作受限的线性表。学习本章之前需要先掌握线性表的顺序存储与链式存储，因为栈和队列的两种实现方式都直接沿用这两种存储结构。本章内容分为三部分：3.2 节介绍栈，3.3 节介绍队列，3.4 节和 3.5 节讨论它们在括号匹配、表达式求值、递归和排队服务中的应用。

3.2 栈

3.2.1 栈的定义

栈（又称堆栈，英文 stack）是只允许在一端进行插入和删除操作的线性表。允许操作的一端称为栈顶，另一端称为栈底。不含任何元素的栈称为空栈。由于最后进入的元素最先被删除，栈的操作特性可以概括为后进先出（Last In First Out, LIFO）。

栈的基本操作包括初始化、判空、入栈、出栈和读栈顶元素。入栈（Push）是在栈顶插入一个新元素，使之成为新的栈顶；出栈（Pop）是删除栈顶元素并返回它。入栈和出栈都只涉及栈顶，因此不论栈中有多少元素，这两个操作都可以在常数时间内完成。

3.2.2 顺序栈

采用顺序存储的栈称为顺序栈。顺序栈用一个数组 `data[0..MaxSize-1]` 存放元素，并用整型变量 `top` 指示栈顶元素的下标。约定空栈时 `top = -1`；入栈时先令 `top` 加 1 再写入元素，出栈时先读出元素再令 `top` 减 1。

当栈中已有 `MaxSize` 个元素时再执行入栈，就会发生上溢；对空栈执行出栈则发生下溢。上溢说明预分配的空间不足，是顺序栈的主要缺点。为了提高空间利用率，可以让两个栈共享同一个数组，两个栈底分别设在数组两端，栈顶相向增长，这种结构称为共享栈。只有当两个栈顶相遇时，共享栈才会发生上溢。

3.2.3 链栈

采用链式存储的栈称为链栈。链栈通常用不带头结点的单链表实现，并规定所有操作都在表头进行，表头指针就是栈顶指针。链栈的优点是不存在上溢问题，便于多个栈共享存储空间；代价是每个结点需要额外的指针域。

顺序栈与链栈相比，两者的入栈、出栈时间复杂度都是 $O(1)$ ；顺序栈需要预先估计容量，链栈则按需申请结点。

3.2.4 出栈序列的性质

设有 n 个元素按 $1, 2, \dots, n$ 的顺序依次入栈，入栈与出栈可以任意交错，于是可能得到多种不同的出栈序列。

****性质 3.1（合法出栈序列判定）****：一个排列 $p_1 p_2 \dots p_n$ 是合法出栈序列，当且仅当不存在下标 $i < j < k$ 使得 $p_j < p_k < p_i$ 。也就是说，若某个较大的元素先出栈，则比它小且仍在栈中的元素必须按从大到小的顺序出栈。

****定理 3.1（出栈序列计数定理）****： n 个不同元素依次入栈，可能得到的不同出栈序列的个数等于第 n 个卡特兰数。

卡特兰数公式为 $C_n = C(2n, n) / (n + 1)$ ，其中 $C(2n, n)$ 表示从 $2n$ 个元素中取 n 个的组合数。例如 $n = 3$ 时， $C_3 = 20 / 4 = 5$ 。

****例 3-1（出栈序列判断）****：元素 $1, 2, 3$ 依次入栈，判断 $3\ 1\ 2$ 是否为合法出栈序列。 3 最先出栈，说明 1 和 2 此时都在栈中且 2 在 1 之上，下一个出栈的只能是 2 而不能是 1 ，所以 $3\ 1\ 2$ 不合法。按性质 3.1，取 $i = 1, j = 2, k = 3$ ，有 $p_2 = 1 < p_3 = 2 < p_1 = 3$ ，同样得出不合法的结论。全部 5 个合法序列为 $123, 132, 213, 231, 321$ ，这与定理 3.1 的计数结果一致。

3.3 队列

3.3.1 队列的定义

队列（queue）也是一种操作受限的线性表，它只允许在一端插入、在另一端删除。允许插入的一端称为队尾，允许删除的一端称为队头。最先进入队列的元素最先离开，因此队列的操作特性是先进先出（First In First Out, FIFO）。先进先出与栈的后进先出恰好相反，二者常被放在一起对照学习。

队列的基本操作包括初始化、判空、入队、出队和读队头元素。入队是在队尾插入元素，出队是删除队头元素。

3.3.2 循环队列

用数组实现队列时，设置队头指针 $front$ 和队尾指针 $rear$ 。若每次出队都只让 $front$ 后移，数组前部腾出的空间将无法再利用，可能出现 $rear$ 已到数组末尾而数组前部仍有空位的情况，这种现象称为假溢出。

为了解决假溢出，把数组在逻辑上视为首尾相接的环，这样的队列称为循环队列。理解循环队列需要先掌握取模运算，因为指针的回绕完全依靠取模实现。指针后移公式为： $\text{rear} = (\text{rear} + 1) \% \text{MaxSize}$ ，front 的后移方式相同。

循环队列中元素个数的计算公式为： $(\text{rear} - \text{front} + \text{MaxSize}) \% \text{MaxSize}$ 。

****性质 3.2（循环队列判空与判满）****：约定 front 指向队头元素、rear 指向队尾元素的下一个位置，并牺牲一个存储单元，则队空的条件是 $\text{front} == \text{rear}$ ，队满的条件是 $(\text{rear} + 1) \% \text{MaxSize} == \text{front}$ 。牺牲一个单元是为了区分队空和队满这两种状态；也可以改用计数变量 size 来区分。

****例 3-2（循环队列指针变化）****：设 $\text{MaxSize} = 6$ ，初始 $\text{front} = \text{rear} = 0$ 。依次让 a、b、c、d、e 入队后 $\text{rear} = 5$ ，此时 $(5 + 1) \% 6 = 0$ 等于 front，队列已满，最多只能存放 5 个元素。接着出队两次， $\text{front} = 2$ ；再让 f 入队， $\text{rear} = (5 + 1) \% 6 = 0$ ，指针回绕到数组开头。此时元素个数为 $(0 - 2 + 6) \% 6 = 4$ ，正是 c、d、e、f 四个元素。

3.3.3 链队列

采用链式存储的队列称为链队列，它是同时带有队头指针和队尾指针的单链表。入队在链表尾部插入结点，出队删除链表的第一个结点。链队列不会产生假溢出，也不需要预先确定容量，适合元素个数变化较大的场合。

3.3.4 双端队列

双端队列是允许在两端都进行插入和删除的线性表。如果限制双端队列只在一端插入和删除，它就退化为栈；如果限制只在一端插入、另一端删除，它就退化为普通队列。因此双端队列与栈、队列都密切相关。

3.3.5 栈与队列的比较

从逻辑结构看，栈和队列与线性表完全相同，都是元素之间一对一的线性关系；二者的区别只在于允许进行插入和删除的位置不同。栈的插入和删除都在同一端完成，所以最后到达的元素总是最先被处理；队列的插入和删除分别在两端完成，所以最早到达的元素总是最先被处理。选择使用哪一种结构，关键是看问题要求的处理次序：凡是需要“暂存后逆序处理”的场合，例如撤销操作、回溯搜索和函数调用，适合用栈；凡是需要“按到达先后依次处理”的场合，例如任务排队、消息缓冲和逐层访问，适合用队列。

从存储结构看，二者都可以采用顺序存储或链式存储。顺序存储的优点是结构简单、访问局部性好，缺点是容量需要事先确定；链式存储能够按需扩展，但每个结点都要额外保存指针。实际编程时，如果元素个数的上限可以预先估计，通常优先选用顺序实现；如果元素个数波动很大或无法估计，则选用链式实现更为稳妥。

3.4 栈的应用

3.4.1 括号匹配

括号匹配算法基于栈的后进先出特性：从左到右扫描表达式，遇到左括号就入栈；遇到右括号时，若栈为空或栈顶的左括号与之类型不符，则匹配失败，否则弹出栈顶。扫描结束后栈为空，才表示所有括号都匹配成功。

****例 3-3（括号匹配）****：检查串 $[(\text{I})]$ 时，读到 $]$ 时栈顶是 $($ ，类型不符，匹配失败；而串 $\{[()]\}$ 扫描结束时栈为空，匹配成功。

3.4.2 中缀表达式与后缀表达式

日常书写的算术表达式把运算符放在两个操作数中间，称为中缀表达式，例如 $a + b * c$ 。中缀表达式需要依靠运算符优先级和括号才能确定计算次序。把运算符放在两个操作数之后的写法称为后缀表达式，也叫逆波兰式，例如 $a \ b \ c \ * \ +$ 。后缀表达式不含括号，计算次序完全由运算符出现的位置决定，适合计算机处理。

****中缀转后缀算法****：从左到右扫描中缀表达式，操作数直接输出；遇到左括号入栈；遇到右括号则依次弹出并输出栈顶运算符，直到弹出左括号为止；遇到运算符时，先把栈中优先级大于或等于它的运算符依次弹出并输出（遇到左括号停止），再把它入栈。扫描结束后，把栈中剩余运算符依次弹出。该算法需要先理解运算符优先级，否则无法决定何时弹栈。

****后缀表达式求值算法****：从左到右扫描后缀表达式，遇到操作数就入栈；遇到运算符就从栈中弹出两个操作数，先弹出的是右操作数，后弹出的是左操作数，计算后把结果压回栈中。扫描结束时栈中唯一的元素就是表达式的值。掌握后缀表达式的写法是学习后缀表达式求值算法的前提。

表达式求值通常分两步完成：先用中缀转后缀算法得到后缀表达式，再用后缀表达式求值算法算出结果。

****例 3-4（表达式求值）****：求中缀表达式 $(3 + 4) * 2 - 6 / 3$ 的值。先转换为后缀表达式 $3 \ 4 \ + \ 2 \ * \ 6 \ 3 \ / \ -$ 。然后求值：3、4 入栈；遇 $+$ 弹出 4 和 3 得 7 入栈；2 入栈；遇 $*$ 得 14；6、3 入栈；遇 $/$ 得 2；遇 $-$ 计算 $14 - 2$ 得 12。最终结果为 12。

3.4.3 递归与栈

递归是指函数在执行过程中直接或间接调用自身。每进入一层递归调用，系统就把当前层的返回地址、实参和局部变量作为一个工作记录压入递归工作栈；该层返回时再弹出这一记录。递归工作栈是栈在程序运行时的典型应用，理解递归的执行过程要以栈为基础。

递归算法结构清晰，但层数过深时递归工作栈会占用大量空间，甚至导致栈溢出。借助显式的栈保存待处理的状态，可以把递归算法改写为非递归算法，这种做法称为递归转非递归。

****例 3-5（阶乘的递归计算）****：计算 $3!$ 时， $\text{fact}(3)$ 调用 $\text{fact}(2)$ ， $\text{fact}(2)$ 调用 $\text{fact}(1)$ ，递归工作栈中依次压入三层记录； $\text{fact}(1)$ 返回 1 后逐层弹出，依次得到 $\text{fact}(2) = 2$ 和 $\text{fact}(3) = 6$ 。

3.5 队列的应用

队列适合处理需要按到达次序服务的问题，例如操作系统中的作业调度、打印机缓冲区以及图的广度优先遍历。在这些场合中，请求到达的速度和被处理的速度往往不一致，队列起到了缓冲作用：先到的请求先被处理，后到的请求在队尾等待，既不会丢失，也不会被插队。

****例 3-6（银行排队模拟）****：银行只有一个窗口，顾客按到达时间依次入队，窗口空闲时让队头顾客出队办理业务。设顾客 A、B、C 分别在第 0、1、2 分钟到达，每人办理 3 分钟，则 A 在第 0 分钟开始办理，B 等待到第 3 分钟，C 等待到第 6 分钟，平均等待时间为 $(0 + 2 + 4) / 3 = 2$ 分钟。这个例子体现了队列先进先出的服务次序。

3.6 本章小结

栈和队列都是操作受限的线性表：栈限定在栈顶操作，具有后进先出特性；队列在队尾插入、队头删除，具有先进先出特性。二者都有顺序和链式两种实现。循环队列通过取模运算解决了假溢出问题。栈的典型应用包括括号匹配、表达式求值和递归，队列的典型应用包括排队服务与广度优先遍历。